

O MANUAL DO CIENTISTA DE DADOS: DA EXPLORAÇÃO AO PRÉ-PROCESSAMENTO

PARTE 1: ANÁLISE EXPLORATÓRIA DE DADOS (EDA)

Mentalidade desta fase: O objetivo não é limpar ou alterar os dados estruturais ainda, mas entender o comportamento, a anatomia e as falhas do seu conjunto de dados brutos antes de tomar qualquer decisão regulatória.

Passo 1: Investigar a Anatomia do Alvo (Target)

Antes de olhar para as causas, olhe para o efeito. A variável alvo dita o comportamento estatístico do modelo.

- Avalie as estatísticas descritivas e a distribuição visual do seu alvo
- Identifique se a distribuição é simétrica ou assimétrica (*skewed*)
- Se for muito assimétrica, uma transformação logarítmica pode ajudar

```
```python id="p1" import numpy as np import pandas as pd import matplotlib.pyplot as plt

print(df['target'].describe())

df['target'].hist(bins=50) plt.title('Distribuição Original do Alvo') plt.show()

np.log(df['target']).hist(bins=50) plt.title('Distribuição com Transformação Logarítmica') plt.show()
```

💡 **\*\*Dica de Mestre:\*\*** Se o alvo tiver cauda longa à direita, o log estabiliza variância e melhora modelos lineares.

---

### Passo Extra: Analisar o Target em Relação às Variáveis

Entender o comportamento do alvo em relação às variáveis revela poder preditivo.

#### ♦ Cateóricas

```
```python id="p2"
print(df.groupby('coluna_categorica')['target'].mean().sort_values())
```

```
df.groupby('coluna_categorica')['target'].mean().plot(kind='bar')
```

♦ Numéricas

```
```python id="p3" import seaborn as sns
```

```
sns.scatterplot(x=df['coluna_numerica'], y=df['target'])
```

```
df['faixa'] = pd.qcut(df['coluna_numerica'], q=10, duplicates='drop') df.groupby('faixa')
['target'].mean().plot(kind='bar')
```

💡 **\*\*Para classificação binária:\*\***

```
```python id="p4"  
print(df.groupby('coluna_categorica')['target_binario'].mean())
```

💡 **Insight:** Se o target muda entre grupos, a variável tem valor preditivo.

Passo 2: A Divisão da Tribo (Tipagem de Variáveis)

- Separe categóricas e numéricas
- Inclua categorias “disfarçadas” (IDs, códigos)
- Converta para tipo `'O'`
- Separe discretas e contínuas
- Identifique temporais

```
```python id="p5" cat_vars = [col for col in df.columns if df[col].dtype == 'O']
```

```
cat_vars = cat_vars + ['MSSubClass'] df[cat_vars] = df[cat_vars].astype('O')
```

```
num_vars = [col for col in df.columns if col not in cat_vars and col != 'target']
```

```
discrete_vars = [col for col in num_vars if len(df[col].unique()) < 20] cont_vars = [col for col in num_vars if col
not in discrete_vars]
```

```
year_vars = [col for col in num_vars if 'Year' in col or 'Ano' in col or 'Data' in col]
```

```

```

```
Passo 3: Mapear os "Buracos" (Dados Faltantes)
```

```
- Identifique colunas com NA
```

- Calcule proporção
- Crie indicador
- Teste impacto no target

```
```python id="p6"
vars_with_na = [col for col in df.columns if df[col].isnull().sum() > 0]

print(df[vars_with_na].isnull().mean().sort_values(ascending=False))

tmp = df.copy()
tmp['na_indicador'] = np.where(tmp['coluna_teste'].isnull(), 1, 0)

print(tmp.groupby('na_indicador')['target'].mean())
```

💡 **Dica de Mestre:** Se o target muda com ausência, isso é informação.

💡 **Insight extra:** Missing pode representar comportamento, não erro.

Passo 4: Detectar Raridades e Assimetrias Extremas

```
```python id="p7"
proporcoes = df['coluna_categorica'].value_counts() / len(df)
raras = proporcoes[proporcoes < 0.01].index
```

```
zeros_pct = (df['coluna_numerica'] == 0).mean()
```

```

Passo Extra: Tratar Variáveis Muito Assimétricas

```python id="p8"
skewness = df[cont_vars].skew().sort_values(ascending=False)

skewed_vars = skewness[skewness > 1].index

for var in skewed_vars:
    df[var] = np.log1p(df[var])
```

💡 `log1p` evita erro com zero

💡 Yeo-Johnson funciona com negativos

Passo Extra: Tratamento de Dados Temporais

⚠ Nunca embaralhe dados temporais.

```
python id="p9" df = df.sort_values('data')
```

```
split = int(len(df) * 0.8) train = df.iloc[:split] test = df.iloc[split:]
```

```
#### Feature Engineering temporal

python id="p10"
df['ano'] = df['data'].dt.year
df['mes'] = df['data'].dt.month
df['dia_semana'] = df['data'].dt.weekday

df['lag_1'] = df['target'].shift(1)
df['media_7'] = df['target'].rolling(window=7).mean()
```

💡 **Regra de ouro:** o futuro nunca pode influenciar o passado.

PARTE 2: ENGENHARIA DE DADOS (FEATURE ENGINEERING)

Mentalidade desta fase: Transformar dados sem vazamento de informação.

Passo 1: A Blindagem Sagrada (Split Treino e Teste)

⚠ **Erro comum:** usar estatísticas antes do split invalida o modelo.

```
python id="p11" from sklearn.model_selection import train_test_split
```

```
SEED = 42 np.random.seed(SEED)
```

```
X = df.drop(['target'], axis=1) y = df['target']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1, random_state=SEED)
```

```
y_train = np.log(y_train) y_test = np.log(y_test)
```

```
---
```

```
#### Passo 2: Imputação Inteligente de Dados Faltantes
```

```

```python id="p12"
CATEGÓRICAS
X_train['coluna_cat'] = X_train['coluna_cat'].fillna('Missing')
X_test['coluna_cat'] = X_test['coluna_cat'].fillna('Missing')

moda = X_train['coluna_cat_pouco_na'].mode()[0]

X_train['coluna_cat_pouco_na'] = X_train['coluna_cat_pouco_na'].fillna(moda)
X_test['coluna_cat_pouco_na'] = X_test['coluna_cat_pouco_na'].fillna(moda)

NUMÉRICAS
mediana = X_train['coluna_num'].median()

X_train['coluna_num_na'] = np.where(X_train['coluna_num'].isnull(), 1, 0)
X_test['coluna_num_na'] = np.where(X_test['coluna_num'].isnull(), 1, 0)

X_train['coluna_num'] = X_train['coluna_num'].fillna(mediana)
X_test['coluna_num'] = X_test['coluna_num'].fillna(mediana)

```

### Passo 3: Lapidação de Variáveis Numéricas e Temporais

```

```python id="p13" import scipy.stats as stats

```

```

X_train['coluna_continua'] = np.log(X_train['coluna_continua'])
X_test['coluna_continua'] = np.log(X_test['coluna_continua'])

```

```

X_train['coluna_area'], lmbda = stats.yeojohnson(X_train['coluna_area'])
X_test['coluna_area'] = stats.yeojohnson(X_test['coluna_area'], lmbda=lmbda)

```

```

X_train['coluna_assimetrica'] = np.where(X_train['coluna_assimetrica'] == 0, 0, 1)
X_test['coluna_assimetrica'] = np.where(X_test['coluna_assimetrica'] == 0, 0, 1)

```

```

X_train['Tempo_Decorrido'] = X_train['Ano_Fim'] - X_train['Ano_Inicio']
X_test['Tempo_Decorrido'] = X_test['Ano_Fim'] - X_test['Ano_Inicio']

```

```

---
```

```

#### Passo 4: Codificação de Categorias (Encoding Monotônico)

```

💡 Ordenação baseada no target melhora modelos lineares.

```

```python id="p14"
proporcao = X_train['coluna_texto'].value_counts() / len(X_train)

```

```

freq = proporcao[proporcao >= 0.01].index

X_train['coluna_texto'] = np.where(
 X_train['coluna_texto'].isin(freq),
 X_train['coluna_texto'],
 'Rare'
)

X_test['coluna_texto'] = np.where(
 X_test['coluna_texto'].isin(freq),
 X_test['coluna_texto'],
 'Rare'
)

tmp = pd.concat([X_train, y_train], axis=1)

ordenadas = tmp.groupby('coluna_texto')['target'].mean().sort_values().index

mapa = {cat: i for i, cat in enumerate(ordenadas)}

X_train['coluna_texto'] = X_train['coluna_texto'].map(mapa)
X_test['coluna_texto'] = X_test['coluna_texto'].map(mapa)

```

## Passo 5: Escalonamento Final (Scaling) e Exportação

⚠ Nunca ajustar scaler no dataset completo.

```

```python id="p15" from sklearn.preprocessing import MinMaxScaler import joblib

scaler = MinMaxScaler() scaler.fit(X_train)

X_train = pd.DataFrame(scaler.transform(X_train), columns=X_train.columns) X_test =
pd.DataFrame(scaler.transform(X_test), columns=X_train.columns)

X_train.to_csv('X_train_final.csv', index=False) X_test.to_csv('X_test_final.csv', index=False)

joblib.dump(scaler, 'minmax_scaler.joblib') ```

```

⚠ Erros Comuns

- Fazer feature engineering antes do split
- Usar estatísticas globais (*data leakage*)
- Ignorar categorias raras

- Não tratar assimetria
 - Aplicar scaling antes da divisão
 - Em dados temporais: misturar passado e futuro
-

Conclusão

Seguindo esse fluxo você garante:

- Dados bem compreendidos
- Features robustas
- Pipeline reproduzível
- Modelos mais estáveis